

Documentație Tehnică Aprofundată Library Management System (LMS)

Această documentație detaliază exhaustiv arhitectura, modulele, ierarhiile de clase, baza de date, interfața grafică și conceptele avansate care stau la baza platformei Library Management System.

1. Arhitectura de Bază și Design Patterns

Sistemul a fost proiectat modular, având un nucleu central (**Core Backend**) perfect izolat de layer-ul de prezentare vizuală. Această decuplare face ca sistemul să funcționeze impecabil atât cu o interfață vizuală bazată pe Qt, cât și într-un terminal (CLI).

- **Single Source of Truth (Repository Pattern):** Clasa `Biblioteca` este singura care comunică cu fișierul `sqlite3` și menține vectorii principali în memorie. Celelalte clase nu execută query-uri SQL direct.
- **Smart Pointers (`std::shared_ptr`):** Pentru a evita memory-leaks, toate instanțele polimorfice (Utilizatori și Cărți) sunt alocate dinamic și gestionate prin contoare de referințe interne (`shared_ptr`).

- **Polimorfism și Dynamic Cast:** Deoarece o colecție abstractă (`vector<shared_ptr<Carte>>`) conține tipuri mixte, se folosește `std::dynamic_pointer_cast<CarteFizica>` pentru a accesa și modifica atribute specifice la runtime, precum mutarea unei cărți fizice de pe un raft pe altul.

2. Persistența Datelor și Baza de Date

Sistemul folosește baza de date locală **SQLite3**. Datele sunt aduse în memorie la pornirea aplicației (în metoda `incarcaDate()`) și sincronizate tranzacțional.

Tabel / Structură SQLite	Descriere Detaliată
Utilizatori	Stochează toți conturile din sistem. Conține flag-uri de tip și o coloană <code>activ</code> pentru 'Soft Delete'. Un utilizator "șters" rămâne în DB pentru consistența logurilor, dar nu se mai poate loga.
Carti	Fiecare carte este serializată. Elementele generice ocupă coloane specifice, în timp ce elementele distincte (cum ar fi link-ul audio) sunt salvate în câmpuri polimorfice formate prin <code>formatFisier()</code> .

Imprumuturi & Istoric	Conectează <code>ID_Cititor</code> cu <code>ISBN_Carte</code> . Reține data generării și termenul limită.
Loguri (JurnalAudit)	O tabelă de tip Write-Only (Incoruptibilă). Fiecare acțiune de business inserează o linie de log cu <code>data_ora</code> exactă și utilizatorul care a făcut-o.

3. Modulul de Securitate și Autentificare

Securitatea datelor sensibile este o prioritate. Clasa abstractă `Utilizator` expune metode statice de criptare.

```
std::string Utilizator::xorEncode(const std::string
&text) {
    std::string key = "S3cr3t_K3y";
    std::string result = text;
    for (size_t i = 0; i < text.size(); i++) {
        result[i] = text[i] ^ key[i % key.size()];
    }
    return result;
}
```

Procesul decurge astfel:

1. Parola introdusă de utilizator este trecută prin masca XOR.
2. Deoarece XOR produce caractere non-printabile, rezultatul este encodat binar în hexazecimal prin `toHex()`.

3. Șirul hexazecimal (ex: `4A3B12...`) este stocat în baza de date. La autentificare, procesul se validează prin compararea hash-urilor.

4. Simulare Temporală și Gestiune Financiară

Deoarece este greu să testezi funcționalități ce țin de scurgerea anilor, `Biblioteca` utilizează o variabilă `offset_timp` care translatează prezentul real în viitor.

- **Bugetul Central:** Reprezintă portofelul bibliotecii. Se crește din amenzi și scade la plata salariilor. Modificat doar de Director prin `achizitioneazaExemplare` sau `platesteSalarii()`.
- **Sistem de Amenzi:** Funcția `calculeazaZileIntarziere()` verifică timpul virtual. Cititorii primesc amenzi automat care generează un obiect `PlataInAsteptare` la plată, ce trebuie aprobat manual.

5. Structurile de Date Complexe (Ierarhia `Carte`)

Toate resursele implementează interfața abstractă `Carte`.

- `CarteFizica`: Include o structură complexă `Locatie` (Cladire, Sala, Raft). Bibliotecarii folosesc funcția `mutaCarte(isbn, locatie_noua)` pentru a reloca resursele. Stocul fizic este controlat strict și poate fi afectat de degradare. Cărțile defecte se trimit spre reparații la Ingrijitor prin `reparaCarte()`.

- **CarteDigitala**: Moștenește clasa de bază dar suprascrie `getStocDisponibil()` returnând mereu o valoare mare (ex: 999), eliminând astfel conceptul de limită fizică. Are format (PDF/ePub) și URL de descărcare.
- **CarteAudio**: Include durată de ascultare în minute și detalii despre narator. La fel ca cea digitală, stocul este infinit.

6. Gestiunea Rolurilor (Ierarhia `Utilizator`)

Modelul **RBAC (Role-Based Access Control)** se reflectă în implementarea claselor derivate din **Utilizator**.

Clasa	Funcții și Proprietăți Specifice
Director	Proprietăți: <code>departament</code> , <code>salariu</code> , <code>nivel_acces</code> . Metode: Administrarea sistemului, poate șterge definitiv utilizatori și aproba achiziții uriașe din buget.
Bibliotecar	Proprietăți: <code>sectie</code> , <code>salariu</code> , <code>program_lucru</code> . Are funcții operaționale zilnice.

Îngrijitor	Proprietăți: <code>zona</code> , <code>echipament</code> . Rulează reparații fizice ale inventarului deteriorat.
Cititor	Proprietăți: <code>tip_abonament</code> , <code>max_carti</code> . Consumator de conținut cu limite setate în funcție de abonamentul său (ex: VIP vs Standard).

7. Structuri de Date Auxiliare (Workflow-ul Sistemului)

Interacțiunea între entități folosește structuri de tranziție (buffer states):

- `ReturnareInAsteptare`: Când un Cititor apasă "Returnează", cartea nu dispare din contul său, ci intră într-o coadă. Un bibliotecar trebuie să cheme `confirmaReturnare()` după ce inspectează cartea fizic la Tonomat. Dacă o găsește ruptă, cheamă `refuzaReturnareDefecta()` care penalizează cititorul și trimite cartea la Îngrijitor.
- `Rezervare` (Holds): Sistem automat care, prin funcția `verificaRezervariExpirate()`, eliberează rezervările neridicate la timp de cititori.
- `Recenzie`: Stochează `isbn`, `id_cititor`, un `rating (1-5)` și text. Sistemul calculează media stelelor prin `calculeazaRatingMediu()`.

- **Notificare**: Un mecanism intern Push în care **mesajul** este afișat la următorul login al utilizatorului pe dashboard.

8. Interfața Grafică Qt (Frontend)

Logica vizuală din Desktop App este găzduită în `QtMainWindow` și `QtDialogs`.

- **QStackedWidget**: Sistemul folosește Widget-uri Stivuite. Indexul 0 reprezintă ecranul de Login, indexul 1 este interfața de aplicație (`m_appWidget`).
- **Dynamic View Building**: Metoda `buildDynamicAppView()` șterge complet layout-urile vechi la fiecare autentificare și le desenează de la zero, plasând în Sidebar doar butoanele permise de rezultatul polimorfic `u->getRol()` (ex: Directorul va primi buton de "Finanțe", Cititorul nu).
- **Paginarea Catalogului**: Deoarece o bază de date masivă încetinește UI-ul, Catalogul randează doar un număr fix de cărți pe pagină, folosind contoarele `m_catalogPage`, re-executând filtrarea (`applyCatalogFilters()`) de fiecare dată când utilizatorul dă "Next".

9. Interfața Terminal / TUI (CLI)

A doua componentă a arhitecturii este Terminal User Interface, din fișierele `ConsoleUI.cpp` și `ui_functii.cpp`. Aici nu se folosesc butoane reale. Se implementează un sistem bazat pe meniuri numerice și coduri de evadare ANSI (ex. `\033[31m` pentru culori, `std::setw` pentru aliniere). Această componentă

este deosebit de utilă pentru debug sau sisteme server (headless) unde serverul de display Linux X11/Wayland nu este disponibil.